

Ghost Team — Never-Miss Phone Alerts (Claude Watch Setup)

How B's setup pings his phone for every trade, halt, and breach — two independent channels, so a dead app never means a missed alert. Replicate on your own box in ~15 minutes.

The architecture (what you're building)

A Claude Code session on your always-on home machine watches your bot's log on the VPS through a persistent ssh tail. Every trade line wakes Claude instantly; it analyzes the trade against your database and pings your phone twice per trade (entry alert within seconds, verdict alert ~90s later) over two independent channels:

Channel 1 — Claude mobile app push. Fastest. Works whenever the Claude app on your phone is connected to the session. Weakness: if iOS/Android suspends or kills the app, pushes bounce.

Channel 2 — Telegram backup. A tiny helper on your VPS sends through your bot's existing Telegram bot token. Independent of the Claude app entirely — if Telegram works on your phone, the alert lands. Claude uses this automatically whenever Channel 1 bounces, and uses BOTH for critical events (halts, balance-floor approaches, floor-breach fills).

Step 1 — Phone side (5 minutes)

1. Install the Claude mobile app and sign into the same account as your desktop Claude Code.
2. Keep the app alive in the background: do NOT force-swipe it closed, enable Background App Refresh for it, avoid Low Power Mode when you want alerts. (This is what kills Channel 1 — learned live.)
3. Make sure your bot's Telegram alerts already reach your phone (most of us have this). If not: create a bot with @BotFather to get a token, message your bot once, then read your chat_id from `api.telegram.org/bot<TOKEN>/getUpdates`. Put both in your bot's .env as TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID.

Step 2 — VPS helper (one file, no new secrets)

Save as `tg_send.py` next to your bot. It reads the Telegram creds your .env already has — the token never travels anywhere.

```
import sys, base64, urllib.request, urllib.parse
env = {}
with open('/path/to/your/bot/.env') as f:
    for line in f:
        line = line.strip()
        if line and not line.startswith('#') and '=' in line:
            k, v = line.split('=', 1); env[k] = v
tok, chat = env.get('TELEGRAM_BOT_TOKEN'), env.get('TELEGRAM_CHAT_ID')
msg = base64.b64decode(sys.argv[1]).decode('utf-8', 'replace') if len(sys.argv) > 1 else 'ping'
data = urllib.parse.urlencode({'chat_id': chat, 'text': '[CLAUDE WATCH] ' + msg}).encode()
r = urllib.request.urlopen(f'https://api.telegram.org/bot{tok}/sendMessage', data, timeout=10)
print('sent' if r.status == 200 else f'failed {r.status}')
```

Step 3 — Home machine script (one file)

Save as `ping_phone.sh` anywhere durable. Base64 keeps any message text safe across the ssh hop. Replace YOUR VPS with your server (keep your IP out of the shared repo — house rules).

```
#!/bin/bash
MSG="${1:-ping}"
B64=$(printf %s "$MSG" | base64 | tr -d '\n')
ssh -o ConnectTimeout=10 root@YOUR_VPS "python3 /path/to/your/bot/tg_send.py $B64"
```

Test it: `bash ping_phone.sh "hello from my Mac"` → message appears in your bot's Telegram chat within ~2 seconds.

Step 4 — Tell your Claude to run the watch

Open a Claude Code session on the home machine and ask for a per-trade watch. The working recipe from B's box:

1. **Persistent log watcher:** Claude arms a background monitor running `ssh -o ServerAliveInterval=30 root@YOUR_VPS "tail -F -n 0 /path/to/bot.log" | grep --line-buffered "SNIPE\|HALTED\|FLOOR-BREACH\|Traceback"` — every matching line wakes Claude within seconds. The keepalive flags matter: silent ssh deaths were our #1 failure mode.
2. **Per trade:** Claude queries your bot's database for the new row, waits for the resolution, and sends entry + verdict alerts (app push first, Telegram fallback, both for critical).
3. **Heartbeat:** every ~25 minutes Claude verifies the bot process, the watcher processes, and your balance vs your floor — so even total silence is verified-healthy, not assumed. Watchers die silently more often than bots do; the heartbeat is what catches it.
4. House rules apply: the watch session is READ-ONLY on your live bot — no config changes without your written plan + approval.

Security notes (non-negotiable)

The helper reads your token from your own .env on your own VPS — never copy tokens to the home machine, never commit `tg_send.py` with a hardcoded token, never put your VPS IP in the shared repo (`my-bot.md` is gitignored for a reason). The scripts above contain zero secrets as written.

Ghost In the System — internal team document. Working live on B's box since 2026-06-12; entry pings land ~2s after the fill, verdicts ~90s later.